

Java Intern Mini■Projects Pack (20 Interactive Assignments)

Console■based, beginner■friendly Java exercises with clear instructions, sample input/output, and stretch goals.

Submission Checklist (applies to all assignments)

- Source code (Git repo link or zip).
- README.md: how to run (Java version + commands), sample I/O, assumptions, edge cases.
- Optional: notes.md with learnings + improvements.

Coding Guidelines

- Avoid a giant main(): use methods/classes.
- Validate input (handle non■numeric, unknown commands, empty input).
- Use meaningful names and consistent formatting.
- Prefer Scanner for input (or BufferedReader if comfortable).

Suggested Rubric (100 points)

- Correctness (40): meets requirements; correct outputs.
- Validation & edge cases (20): handles bad input gracefully.
- Code quality (20): structure, readability, naming, reuse.
- User experience (10): clear prompts, friendly flow.
- README & examples (10): easy to run; sample I/O included.
- Bonus (+10): thoughtful stretch features.

Starter Templates (Copy/Paste)

Basic console loop

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.print("Enter command (or exit): ");
            String line = sc.nextLine().trim();

            if (line.equalsIgnoreCase("exit")) {
                System.out.println("Goodbye!");
                break;
            }
        }
    }
}
```

```

        if (line.isEmpty()) {
            System.out.println("Please enter something.");
            continue;
        }

        // TODO: parse and handle commands
        System.out.println("You entered: " + line);
    }

    sc.close();
}

```

Safe integer read

```

private static int readInt(Scanner sc, String prompt) {
    while (true) {
        System.out.print(prompt);
        String input = sc.nextLine().trim();
        try {
            return Integer.parseInt(input);
        } catch (NumberFormatException ex) {
            System.out.println("Invalid number. Try again.");
        }
    }
}

```

Assignments (20)

Pick 1–3 depending on your timebox. Each assignment includes core requirements and stretch goals.

1) Guess the Number — Hot/Warm/Cold

Objective: Create a guessing game with temperature hints and attempt counting.

What to do

- 1 Generate a random number between 1 and 100.
- 2 Repeatedly prompt the user for a guess until correct.
- 3 Print 'Too high'/'Too low' plus: within 5 → Hot, within 10 → Warm, else → Cold.
- 4 Count attempts; print attempts on success.
- 5 Validate non-numeric input without crashing.

Sample Input

```
50
75
63
61
```

Sample Output

```
I picked a number between 1 and 100.
Enter guess: 50
Too low ❄ Cold
```

```
Enter guess: 75
Too high ■ Warm
```

```
Enter guess: 63
Too high ■ Hot
```

```
Enter guess: 61
Correct! ■ Attempts: 4
```

Stretch (Bonus) Ideas

- Play again loop
- Difficulty levels (1–20, 1–100, 1–1000)
- Session high score

2) Rock Paper Scissors — Best of 5

Objective: User plays CPU until someone wins 3 rounds.

What to do

- 1 Accept input: rock/paper/scissors (case-insensitive).

- 2 CPU randomly chooses one option.
- 3 Determine round winner and track score.
- 4 Stop when user or CPU reaches 3 wins; print final score.
- 5 Validate invalid input and re-prompt.

Sample Input

```
rock
paper
scissors
scissors
rock
```

Sample Output

```
Round 1: You=rock, CPU=scissors -> You win! (1-0)
Round 2: You=paper, CPU=paper -> Draw (1-0)
...
Final: 3-2
```

Stretch (Bonus) Ideas

- Keep match history
- Add 'best of N' selection

3) Cafe Billing System (Mini POS)

Objective: Order items from a menu and print a receipt with tax.

What to do

- 1 Define a menu (e.g., coffee, tea, sandwich, cookie) with prices.
- 2 Loop: user types item name until 'done'.
- 3 Add valid items; reject unknown items politely.
- 4 At the end, print itemized receipt, subtotal, tax (8.25%), total.
- 5 Support case-insensitive item matching.

Sample Input

```
coffee
cookie
sandwich
done
```

Sample Output

```
Added: coffee ($3.50)
Added: cookie ($1.50)
Added: sandwich ($6.25)
```

```
----- RECEIPT -----
coffee      3.50
cookie      1.50
```

```
sandwich 6.25
Subtotal: 11.25
Tax(8.25%): 0.93
Total: 12.18
-----
```

Stretch (Bonus) Ideas

- Quantities: 'coffee 2'
- Discount code
- Remove item command

4) Password Strength Checker + Suggestions

Objective: Score a password and suggest improvements.

What to do

- 1 Read a password string.
- 2 Check: length $\geq$ 8, uppercase, lowercase, digit, special char.
- 3 Score 0–5 and print strength category.
- 4 Print missing requirements and a suggested improved password.

Sample Input

```
hello123
```

Sample Output

```
Strength: 3/5 (Medium)
Missing: Uppercase letter, Special character
Suggestion: Try 'Hello123!'
```

Stretch (Bonus) Ideas

- Block common passwords
- Estimate complexity score

5) To-Do List Manager (Commands)

Objective: Command-driven todo list with IDs and completion.

What to do

- 1 Support commands: add , list, done , remove , exit.
- 2 Store tasks in ArrayList with auto-increment IDs.
- 3 Print tasks as: #id [] desc or #id [X] desc.
- 4 Validate invalid commands and IDs.

Sample Input

```
add Buy milk
```

```
add Finish Java assignment
list
done 1
list
exit
```

Sample Output

```
Added #1: Buy milk
Added #2: Finish Java assignment
```

```
#1 [ ] Buy milk
#2 [ ] Finish Java assignment
```

```
Marked done: #1
```

```
#1 [X] Buy milk
#2 [ ] Finish Java assignment
```

Stretch (Bonus) Ideas

- search
- save/load to file
- sort by completion

6) Log Analyzer (Counts & Patterns)

Objective: Analyze pasted log lines and count INFO/WARN/ERROR.

What to do

- 1 Read lines until 'END'.
- 2 Count lines containing INFO/WARN/ERROR.
- 3 Print summary counts.
- 4 Optional: detect most common error prefix/pattern.

Sample Input

```
INFO User login success
WARN API response slow
ERROR Payment failed for user 102
ERROR Payment failed for user 103
END
```

Sample Output

```
Summary:
INFO: 1
WARN: 1
ERROR: 2
Top error pattern: 'Payment failed'
```

Stretch (Bonus) Ideas

- Parse timestamps

- Group by module
- Write summary to file

7) Mini ATM Simulator (OOP)

Objective: Build an ATM menu with PIN validation and balance updates.

What to do

- 1 Ask for PIN; allow 3 attempts.
- 2 Menu: Check Balance, Deposit, Withdraw, Exit.
- 3 Deposit must be positive; withdraw cannot exceed balance.
- 4 Use Account/ATM classes.

Sample Input

```
1234
2
50
1
3
30
1
4
```

Sample Output

```
PIN accepted ■
Deposited. New balance: 150
Balance: 150
Withdraw success. New balance: 120
Goodbye!
```

Stretch (Bonus) Ideas

- Transaction history
- Lock account after failures

8) Quiz Game (Random Questions)

Objective: Ask 5 random questions from a pool and score answers.

What to do

- 1 Store questions with multiple-choice options and correct answer.
- 2 Randomly pick 5 without repeats.
- 3 Collect answers and compute score.
- 4 Print final score and optionally review incorrect answers.

Sample Input

```
2
```

1
3
2
4

Sample Output

Q1 ... Your answer: 2 -> Correct!
...
Final Score: 4/5

Stretch (Bonus) Ideas

- Add timer
- High scores

9) Text Adventure — Choose Your Path

Objective: Interactive branching story with 3 levels of choices.

What to do

- 1 Print scenario and 2–3 choices (1/2/3).
- 2 Branch to next scenario based on choice.
- 3 Track a simple score (e.g., courage points).
- 4 End after ~3 decision levels and print ending + score.

Sample Input

1
2
1

Sample Output

You wake up in a forest... Choice: 1
...
Ending: You survived! Score: 7

Stretch (Bonus) Ideas

- Inventory items
- Random events

10) Hangman (Word Guess Game)

Objective: Guess letters to reveal a hidden word before attempts run out.

What to do

- 1 Pick a word from a predefined list.
- 2 Display masked underscores and guessed letters.
- 3 Accept one-letter guesses; update state.

- 4 Limit wrong attempts (e.g., 6).
- 5 Prevent duplicate guesses.

Sample Input

a
e
o
t
r

Sample Output

Word: _ _ _ _ _
Wrong left: 6
Guess: a
Word: _ a _ _ _
...
You won! Word was: bacon

Stretch (Bonus) Ideas

- Difficulty by word length
- ASCII hangman art

11) Palindrome + Anagram Checker (String Tools)

Objective: Menu-driven string utility for palindrome/anagram/vowels.

What to do

- 1 Menu: (1) Palindrome (2) Anagram (3) Vowel/Consonant counts (4) Exit.
- 2 Normalize input: lowercase, remove spaces (optionally punctuation).
- 3 Palindrome: compare cleaned string to reverse.
- 4 Anagram: compare sorted character arrays.

Sample Input

1
Never odd or even
2
listen
silent
4

Sample Output

Palindrome? YES ■
Anagram? YES ■

Stretch (Bonus) Ideas

- Ignore punctuation
- Support phrases

12) Dice Battle (Turn-Based Game)

Objective: User vs CPU with HP, random attack/heal actions.

What to do

- 1 Start HP: user=30, CPU=30.
- 2 Each round user chooses attack/heal.
- 3 Attack deals 1–6; heal restores 1–4.
- 4 CPU chooses randomly; apply outcomes.
- 5 Stop when someone reaches 0 HP; print winner.

Sample Input

```
attack
attack
heal
attack
```

Sample Output

```
Round 1: You attack for 5... HP -> You: 27, CPU: 25
...
You WIN! ■
```

Stretch (Bonus) Ideas

- Critical hits
- Defend action

13) Calculator with History (Commands)

Objective: Command-based calculator storing recent results.

What to do

- 1 Commands: add a b, sub a b, mul a b, div a b, history, exit.
- 2 Parse input safely; validate numeric operands.
- 3 Store history strings in ArrayList.
- 4 Handle divide-by-zero gracefully.

Sample Input

```
add 5 7
mul 3 4
history
exit
```

Sample Output

```
Result: 12
Result: 12
History:
1) add 5 7 = 12
```

2) mul 3 4 = 12

Stretch (Bonus) Ideas

- Keep last 10 entries
- Support decimals

14) Flashcards (Study App)

Objective: Q/A flashcards with shuffle and scoring.

What to do

- 1 Create a list of Q/A pairs.
- 2 Shuffle and ask N questions.
- 3 Compare answers (case-insensitive) and track score.
- 4 Print final score and review incorrect answers.

Sample Input

Java Virtual Machine
JIT

Sample Output

Q1: What does JVM stand for? ... Correct ■
Final score: 2/2

Stretch (Bonus) Ideas

- Retry incorrect cards
- Partial match acceptance

15) Poll/Voting App (Live Tally)

Objective: Collect votes for candidates and display results.

What to do

- 1 Define candidates (A, B, C or names).
- 2 Loop read votes until 'done'.
- 3 Validate vote input.
- 4 Print totals, percentages, and winner (handle ties).

Sample Input

A
B
A
C
A
done

Sample Output

Results: A:3 B:1 C:1

Winner: A ■

Stretch (Bonus) Ideas

- Percentages
- Tie-break display

16) Simple Appointment Scheduler

Objective: Add/list/remove appointments sorted by date/time.

What to do

- 1 Commands: add YYYY-MM-DD HH:MM Title | list | remove | exit.
- 2 Store appointments in a list with id, LocalDateTime, title.
- 3 Sort by date/time for listing.
- 4 Validate date/time format and invalid IDs.

Sample Input

```
add 2026-06-01 14:30 Dentist
add 2026-05-29 09:00 Standup
list
```

Sample Output

```
Sorted:
#2 2026-05-29 09:00 Standup
#1 2026-06-01 14:30 Dentist
```

Stretch (Bonus) Ideas

- Prevent overlaps
- Search by keyword

17) Expense Splitter (Group Calculation)

Objective: Compute who owes/gets back to split costs evenly.

What to do

- 1 Ask number of people; read name and amount paid.
- 2 Compute total and average per person.
- 3 For each person print: owes (avg-paid) or gets back (paid-avg).
- 4 Optionally generate minimal transactions to settle.

Sample Input

```
3
Alice 60
```

Bob 30
Cara 0

Sample Output

Each should pay: 30
Alice gets back: 30
Bob settled: 0
Cara owes: 30

Stretch (Bonus) Ideas

- Generate transactions: 'Cara pays Alice 30'
- Read until 'done'

18) Word Frequency Counter (Mini Analytics)

Objective: Count words in text and print top N most frequent.

What to do

- 1 Read multiple lines until 'END'.
- 2 Normalize: lowercase; remove punctuation.
- 3 Split into words and count with HashMap.
- 4 Sort by frequency descending and print top 5.

Sample Input

Hello world! Hello Java.
Java is fun. Hello again.
END

Sample Output

Top words:
hello -> 3
java -> 2
world -> 1
is -> 1
fun -> 1

Stretch (Bonus) Ideas

- Ignore stop-words
- Export to file

19) Traffic Light Simulator (State Machine)

Objective: Simulate RED/GREEN/YELLOW cycles with emergency override.

What to do

- 1 Create enum states: RED, GREEN, YELLOW.
- 2 Cycle with timers: RED 5s, GREEN 5s, YELLOW 2s.

- 3 Print current state with countdown.
- 4 Between cycles, accept command: 'emergency' → switch to RED; 'exit' → stop.

Sample Input

```
emergency  
exit
```

Sample Output

```
Current: GREEN (4)  
Type command: emergency  
■ Emergency detected! Switching to RED  
...
```

Stretch (Bonus) Ideas

- Pedestrian 'walk' button
- Custom durations

20) Race Simulator (Random Progress + Leaderboard)

Objective: Simulate a race among 2–5 players with random movement.

What to do

- 1 Ask number of players and names.
- 2 Track length = 50 units; each round move each player 1–6.
- 3 Print progress using repeated '#'.
4 Stop when someone reaches finish; print winner and optionally rankings.

Sample Input

```
3  
Alice  
Bob  
CPU
```

Sample Output

```
--- Round 1 ---  
Alice: #####  
Bob: ###  
CPU: #####  
...  
■ Winner: Alice ■
```

Stretch (Bonus) Ideas

- Finishing order
- Random boost/penalty events